

Public Sentiment on War-Related Discussions in Australian Social Media and Its Relationship with Oil Prices

COMP90024 Assignment 2 Team 8 Report
COMP90024 Team 8

Team members: Chenxuan Jin, Linhang Yu, Yaojin Wu, Jianjie Wang, Jiayi Luo

20 May 2026

Contents

Abstract	3
1 Project Overview	3
1.1 Background Introduction	3
1.2 Research Objectives	3
2 Teamwork	4
2.1 Group Members and Contribution	4
2.2 GitLab Repository and Demo Video Link	4
3 System Architecture Design	5
3.1 Front-end	5
3.2 Backend	6
3.3 Elasticsearch Storage and Analytics Layer	6
4 Data Acquisition and Processing Pipeline	7
4.1 Web Crawling and Data Collection	7
4.2 Data Cleaning and Preprocessing	7
4.3 Sentiment Analysis and Structuring	8
4.4 Visualization and Interpretation	8
5 Platform Deployment	15
5.1 Docker and Local Development	15
5.2 Kubernetes and NeCTAR/MRC Cloud	15

5.3	Fission and Scheduled Harvesting	17
6	Technology Selection and Evaluation	19
7	Discussion	20
7.1	Suitability of the Cloud-native Architecture	20
7.2	Kubernetes and MRC Trade-offs	20
7.3	Fission, CronJob and Scheduled Ingestion	20
7.4	Elasticsearch and API Design Trade-offs	20
7.5	Data Quality, Australia Relevance and NLP Limitations	21
7.6	Scalability, Fault Tolerance and Future Improvements	21
8	Error Handling, Security and Reliability	22
9	Challenges and Solutions	23
10	Project Summary	24

Abstract

This report presents a cloud-based analytics application for examining Australian-related social media discussion of the Ukraine War, the Gaza Conflict and Iran Tensions, and for comparing the resulting activity and sentiment patterns with official Brent and WTI crude oil price data. The implemented pipeline collects Mastodon and BlueSky posts, retrieves spot price records from the United States Energy Information Administration, cleans and normalises the data, assigns rule-based conflict categories, calculates TextBlob sentiment polarity, stores records in Elasticsearch, exposes them through a FastAPI REST backend, and visualises the results in a Jupyter Notebook frontend.

The saved Notebook dataset contains 195 social media posts collected between 4 October 2025 and 7 May 2026, together with 2175 market price records. The two platforms contribute almost the same number of posts, but most conflict-related posts focus on the Ukraine War. Sentiment in the collected Australia-related posts is mostly neutral to mildly positive across the three conflict themes, with only a small proportion of posts showing negative polarity. Mastodon posts show slightly higher average sentiment scores than BlueSky posts.

Over time, posting activity rises sharply in early May, while average sentiment fluctuates rather than moving in a single direction. The oil-price comparison shows a moderate positive correlation between post count and price, a weak positive correlation between average sentiment and price, and almost no relationship between post count and average sentiment.

The main engineering contribution is the end-to-end cloud analytics path: external data harvesting, preprocessing, Elasticsearch indexing, REST access, Kubernetes deployment, Fission timer-triggered ingestion, an optional Kubernetes CronJob manifest for backup ingestion, and notebook-based visual analytics.

1 Project Overview

1.1 Background Introduction

In 2026, for the Iran War impacted oil trade in the Persian Gulf and the closure of the Strait of Hormuz led to oil prices increased. Kilian et al find that oil prices rise after the war (Kilian et al., 2026). Therefore, international political conflicts can affect the stability of the global energy market. Currently, the war in Ukraine, the Gaza conflict, and tensions with Iran have not truly ended, and these three major conflicts will affect global oil prices. Australian fuel prices are mainly affected by international oil prices and the US dollar exchange rate (Australian Competition and Consumer Commission, n.d.). As these conflicts unfold, oil prices are gradually rising. It will lead to a shift in Australian residents' attitudes toward the Ukraine war, the conflict in Gaza and tensions with Iran.

For the social media has become a crucial platform for public discussion of international conflicts in Australia. Mastodon and Bluesky, as our primary data sources, provide an effective window into public discourse beyond mainstream social media channels. Thus, as oil prices rise, Australians' attitudes towards posts related to the war in Ukraine, the Gaza conflict, and the situation in Iran reflect their emotional responses. This project, against this backdrop, built a cloud-based workflow to collect social media posts, calculate user sentiment, store the processed data in Elasticsearch, and compare observed patterns with official oil price time series data.

1.2 Research Objectives

Therefore, based on the Background Introduction in 1.1, our chosen topic is: What do Australians think of the global conflicts, e.g., the wars in Ukraine, Iran, or what is happening in Gaza? Is one platform more vocal (have more posts) on these events, and is there a change in the sentiment of the population? Will global oil prices affect

Australians’ emotional expression regarding the conflict? Therefore, we decided to investigate whether there are differences in the level of discussion activity among Australians regarding the Ukraine War, Gaza Conflict, and Iran Tensions; whether there are differences in Australians’ attitudes towards the Ukraine War, Gaza Conflict, and Iran-related tensions on the Bluesky and Mastodon platforms; whether Australians’ attitudes towards the Ukraine War, Gaza Conflict, and Iran-related tensions have changed significantly over time; and will Australians’ attitudes toward the war in Ukraine, the conflict in Gaza, and tensions related to Iran change as oil prices rise? This study is based on our analysis of crawled data and does not represent the attitudes of all Australians.

2 Teamwork

2.1 Group Members and Contribution

The information contribution of the members of the team⁸ is shown below:

Name	Contribution
Yaojin Wu (1305882)	focused on data collection and initial cleaning, including Mastodon and Bluesky harvesting, EIA oil price extraction, pagination, keyword searches, duplicate reduction and structured output.
Linhang Yu (1346905)	focused on preprocessing and visualisation, including date standardisation, missing-value handling, text cleaning, TextBlob sentiment scoring, conflict classification and notebook charts.
Chenxuan Jin (1827414)	focused on GitLab repository creation and management, README maintenance, Docker and Elasticsearch setup, backend interface coordination, and local pipeline testing.
Jianjie Wang (1719015)	focused on backend API integration, local and cloud testing, Kubernetes deployment files and README material.
Jiayi Luo (1741434)	focused on project-scope analysis, report writing, technical documentation integration, presentation narrative and final consistency checking across requirements, code, repository files and notebook outputs.

2.2 GitLab Repository and Demo Video Link

The entire project was completed by Yaojin, Linhang, Chenxuan, and Jianjie. All the data and scripts can be found in the GitLab code repository. In the GitLab repository, the backend directory contains the FastAPI service and ingestion code, the fission directory contains the Fission trigger-ingestion function for scheduled ingestion, the k8s directory contains the Kubernetes deployment manifest, the frontend directory contains the Notebook frontend for API-based data access, the data directory contains cleaned and exported data for validation, and the README document records local, cloud, and Fission ingestion workflows with alternatives. There is a video on YouTube showing our group demonstrating the core functionality of the system.

- GitLab Repository: https://gitlab.unimelb.edu.au/chenxuanjin/COMP90024_team_8
- YouTube Video: <https://youtu.be/K9kXgMK1Kuw>

3 System Architecture Design

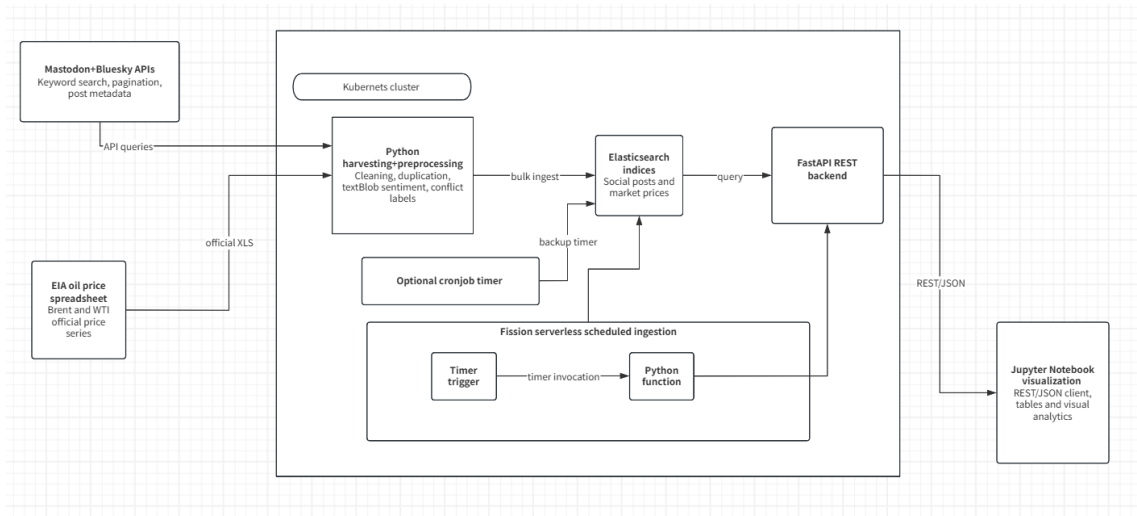


Figure 1: Cloud analytics and scheduled-ingestion architecture

Figure 1 summarises the implemented system architecture design. Mastodon and BlueSky posts enter through platform APIs using keyword search and pagination, while the EIA spreadsheet provides the official oil-price time series. Python harvesting and preprocessing code cleans text, removes invalid or duplicate records, calculates TextBlob sentiment and assigns rule-based conflict labels. The processed documents are inserted into two Elasticsearch indices, social posts and market prices, using bulk ingestion. FastAPI then exposes resource-oriented REST endpoints for posts, search, statistics, keyword market prices, and the Jupyter Notebook consumes those endpoints as JSON for charts and exploratory analysis.

The cloud deployment runs the backend components inside the NeCTAR/MRC Kubernetes environment. Kubernetes provides internal service names, container scheduling, resource declarations and restart behaviour. Fission provides the serverless scheduled-ingestion layer: the ingestion-daily timer invokes the trigger-ingestion Python function, which sends a POST request to the internal FastAPI `/ingest/run` endpoint. A Kubernetes CronJob manifest is documented as an optional backup path that can run `backend/ingest_real_data.py` from the backend Docker image if applied. The Jupyter Notebook acts as an external client of the deployed backend rather than as a service running inside the Kubernetes cluster.

In system, it was organised around shared schemas and stable API contracts rather than isolated scripts. This modular design connected data collection, preprocessing, storage, backend services and notebook visualisation into one pipeline, while also allowing team members to develop different components in parallel.

3.1 Front-end

The frontend is implemented as a Jupyter Notebook visualisation interface. The visual layer focuses on the relationships required by the research questions. It includes a high-frequency word cloud, platform and conflict-topic post counts, sentiment distributions and boxplots, daily activity and sentiment trends, a conflict-platform sentiment heatmap, oil-price time-series comparison, a sentiment-price scatter plot and a correlation heatmap. These charts transform the data retrieved through the backend pipeline into visual outputs that support the analysis of platform activity, topic attention, sentiment differences and the relationship between social discussion and oil-price movements.

3.2 Backend

The architecture for backend can be summarised through two connected pipelines: the main analytics pipeline and the scheduled-ingestion pipeline.

External APIs and EIA spreadsheet -> Python ingestion/preprocessing -> Elasticsearch bulk indexing
-> FastAPI REST endpoints -> Jupyter Notebook visual analytics

Fission timer ingestion-daily -> Fission function trigger-ingestion -> POST /ingest/run -> existing ingestion pipeline -> Elasticsearch

The backend, storage and scheduled-ingestion design follows a layered flow. The FastAPI backend is not only a query service; it also exposes POST /ingest/run so that an in-cluster Fission function can trigger the same ingestion logic used by local and CronJob execution. This design keeps the Fission function small. Instead of duplicating the whole harvester inside Fission, trigger_ingestion.py calls the internal service address `http://comp90024-api.comp90024.svc.cluster.local:8000/ingest/run` and lets the API container run the established ingestion functions against Elasticsearch.

Most backend endpoints are resource-oriented, and /ingest/run is an operational trigger endpoint for scheduled ingestion. /health checks API and Elasticsearch connectivity. /ingest/run performs an ingestion run. /posts returns cleaned social media documents and supports keyword filtering. /posts/search supports explicit keyword search. /stats/platform, /stats/conflict, /stats/sentiment, /stats/platform-sentiment, /stats/timeline, /stats/keywords and /stats/summary provide aggregated views. /market/prices and /market/latest expose official price records. This API contract allows the Notebook to remain a presentation and analysis layer rather than embedding all storage or ingestion logic in notebook cells.

3.3 Elasticsearch Storage and Analytics Layer

Elasticsearch is the central storage and analytics layer in this project. After Mastodon and BlueSky posts and EIA oil price records are collected, cleaned and structured, they are inserted into two Elasticsearch indices: social_posts and market_prices. The social_posts index stores semi-structured social media records, including platform, date, text, author, upvotes, URL, sentiment score, conflict category and cleaned text. The market_prices index stores official Brent and WTI oil price records, including date, benchmark and price fields.

Compared with storing results only in CSV files, Elasticsearch provides a more suitable backend because it supports document-oriented storage, fast keyword search, date-based filtering and aggregation queries. These capabilities support the backend statistical endpoints, such as platform counts, conflict-topic counts, sentiment summaries and time-based analysis. FastAPI retrieves data from Elasticsearch and exposes query and aggregation results through REST endpoints. Finally, use these endpoints for visualization in Jupyter Notebook. Therefore, Elasticsearch can connect data ingestion, backend, and frontend.

4 Data Acquisition and Processing Pipeline

4.1 Web Crawling and Data Collection

The data used in this project were collected from two social media platforms and one official source: Mastodon, Bluesky, and the Energy Information Administration (EIA). Each platform provides a different API, allowing automated data collection to be implemented according to its own structure.

For Mastodon, mastodon.py library was used to access the API and start the data crawling after completing the authentication. Unlike the Reddit approach of combining keywords into a single query, each keyword was searched independently in Mastodon. Although this approach reduces overall coverage to some extent, it improves thematic relevance. The number of posts collected for each keyword was controlled to prevent a single keyword from dominating the results. In order to paginate and avoid duplicate records, the unique ID of each post was used. Since Mastodon post IDs are related to posting order, the code uses post IDs as the basis for pagination, allowing it to retrieve an earlier batch of search results in subsequent requests. For each returned post, BeautifulSoup was applied to remove HTML tags from the content and extract plain text.

For Bluesky, data were collected through its public API endpoints after authentication using an account identifier and password and obtaining an access token. After the login credentials were submitted and the access token was retrieved, adopted the same independent keyword query strategy as in Mastodon. For pagination, search request parameters were constructed to include the current keyword, the maximum number of results returned in a single request, and the current pagination position, enabling continuous multi-page data retrieval.

To prevent overrepresentation of each keyword, limits were applied both to the total amount of data collected from each platform and to the number of posts collected for each keyword. This helps maintain the balance and representativeness of the dataset. Since the data structures differ across platforms, each collected post was processed into a unified structured format by extracting and recording its platform, publication date, text content, author, platform score and corresponding URL. Finally, collected data converted into DataFrame format and integrated into a complete dataset for next analysis.

This project selected oil prices from the U.S. Energy Information Administration (EIA) as the official data source. The code identifies the relevant data location in the spreadsheet by searching for the Brent and WTI price columns and traversing possible header positions. After extraction, the original wide-format table was converted into a long-format table, in which Brent and WTI appear as values in the benchmark column. Since the project aims to analyse the impact of wars on market conditions in recent years, and the three selected conflicts are Iran-related tensions, the Gaza conflict, and the Ukraine war, with the Ukraine war being the earliest and beginning in 2022, only oil price data from 1 January 2022 onward were retained.

4.2 Data Cleaning and Preprocessing

After collecting the post data, next step is process the text data, then generate structured for visualization and analysis. First, we defined the `analyze_sentiment` function to calculate the sentiment score of conflict discussion posts related to Australia on two websites. Through `pd.isna(text)` check for missing values, remove leading and trailing spaces to determine if the text is empty. If it is empty, return 0.0. If the text exists, we use `TextBlob` for sentiment analysis. Finally, we returned a sentiment score.

Categorize conflicting topics in post text using keywords. For the `categorize_conflict` function through `str(text).lower()` converts the text to lowercase. Then, if post appear the words `ukraine`, `russia` or `putin`. It belongs to Ukraine War. If this condition is not satisfied, checks for appear the words `gaza`, `israel` or `palestine`. If any of these appear, it belongs Gaza Conflict. If these two conditions are not satisfied, if appear `iran`. It belongs Iran Tensions. If none

Therefore, data preprocessing mainly includes the following four steps:

- Finally, through `df_all` to summary the data from two platforms. The first step is to standardize the date format, removing invalid dates and maintaining date in days. The second step is to replace all missing values to "", generating sentiment scores and conflict labels. The third step is to delete text about non-relationship with topic and remove duplicate records. Finally get the data.

Sentiment analysis returns values ranging from -1 to 1, with plural values indicating negative attitudes, positive values indicating positive attitudes, and values closer to zero indicating neutrality. Conflict categorization is rule-based. Posts containing Ukraine, Russia or Putin are belong to Ukraine War. Posts containing Gaza, Israel or Palestine belong to Gaza Conflict. Posts containing Iran are belong to Iranian Tensions. Posts that do not match are marked as Other.

[illegible]

Figure 2 shows the high-frequency words across all posts. The large sizes of word in word cloud indicate higher frequency of the word. The words with highest frequency in the word cloud are people, Ukraine, Australia, Israel, Iran, Palestine, resistance, war, and state. These words indicate that discussions are highly focused on issues related

to war, nation, political stance, and affected social groups. Ukraine, Israel, Iran, and Palestinian relate to the three conflict themes of interest in this study, while words such as people, right, world, and state suggest that users not only discuss conflicts as the military events but also connect them to human rights, national stance, international order, and the plight of affected groups. The high frequency of Australia also supports the project's scenario setting, as the dataset aims to capture discussions related to Australia. Furthermore, the word cloud illustrates that war-related social media discussions are not simply information dissemination. When discussing international conflicts, users often connect the events themselves with broader social issues such as national responsibility, the plight of civilians, resistance actions, international support and moral judgment.

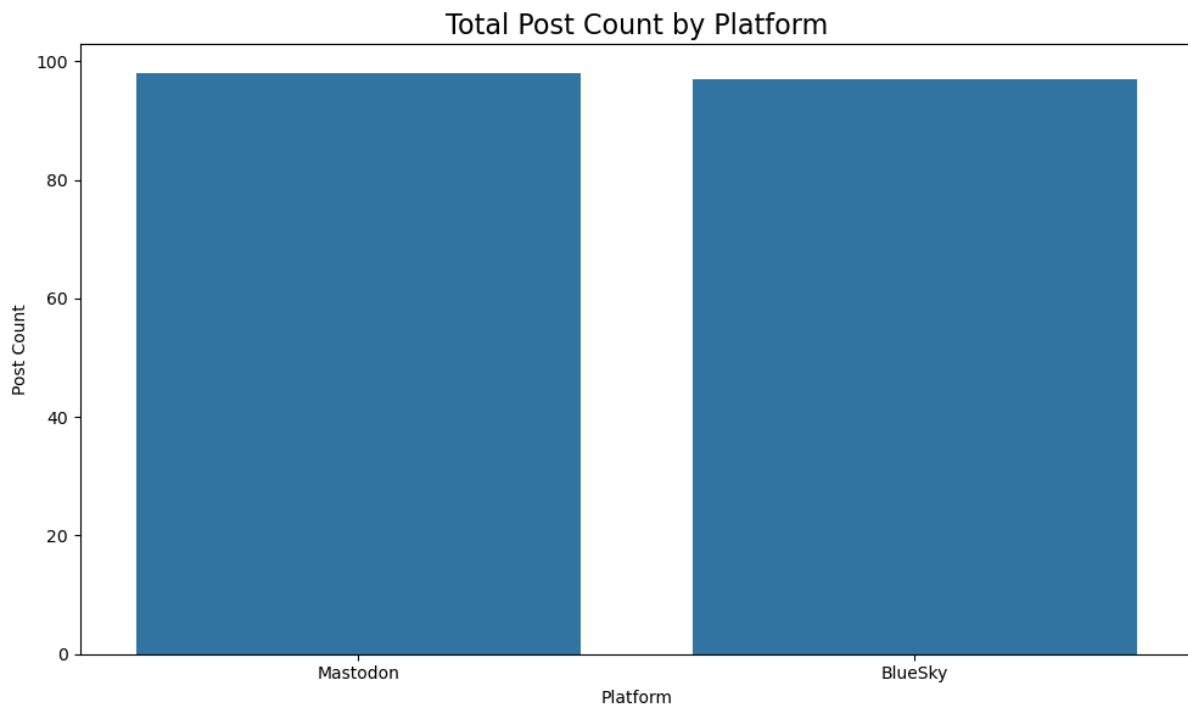


Figure 3: Total post count by platform

Figure 3 shows the number of posts crawled from the Mastodon and Bluesky platforms. The plot shows that the number of posts for two platforms are both close to 100. This indicates that the dataset is balanced in terms of platform-level data crawling. This balance is beneficial for subsequent comparisons the sentiment patterns between two platforms, as the results will not dominated by one platform.

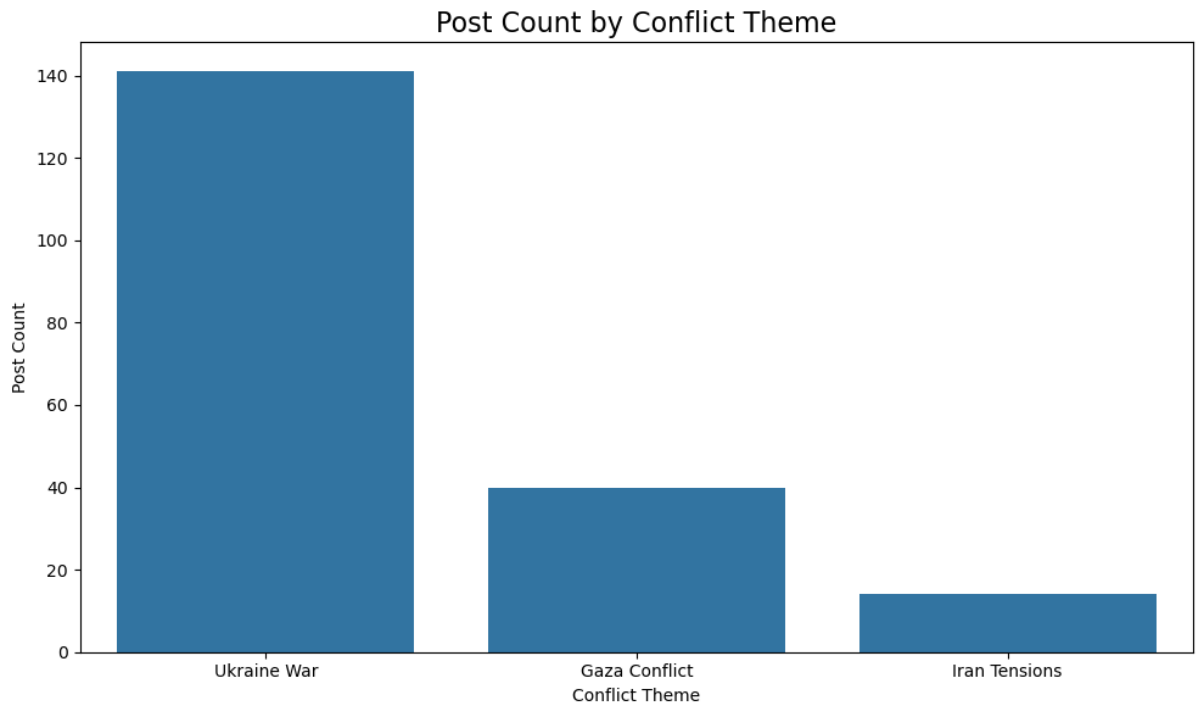


Figure 4: Post count by conflict theme

Figure 4 shows the number of posts crawled across different conflict themes. The plot shows that the Ukraine War crawled around 140 posts, the Gaza Conflict crawled around 40 posts and Iran Tensions crawled 15 posts. The result indicates that the discussion intensity of different conflict themes on social media is uneven in the crawled data, with the Ukraine War receiving the highest level of user discussion. This may be because the Ukraine War has lasted for a longer period of time, leading to higher attention from Australian social media users. In contrast, Iran Tensions has the lowest number of posts, this suggest that this theme has a more limited discussion scope in the dataset. This may be because people only discuss this theme when specific geopolitical events occur.

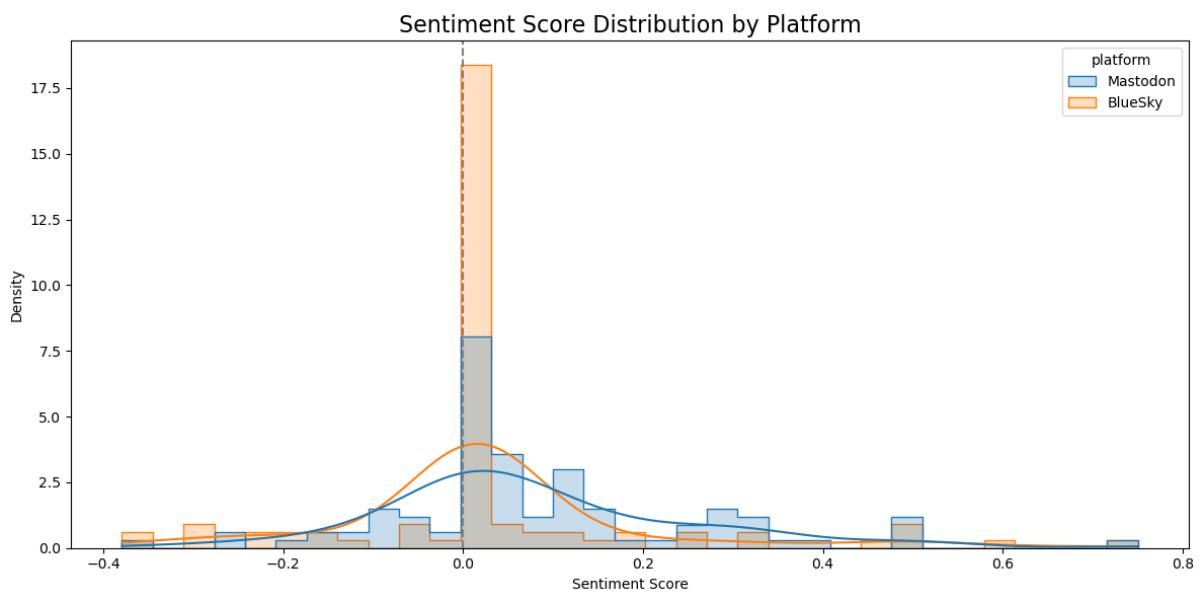


Figure 5: Sentiment score distribution by platform

Figure 5 shows the distribution of sentiment scores on the Mastodon and BlueSky platforms. The plot shows that sentiment scores on both platforms are around zero, indicate that the attitude for most posts are neutral. However, the distribution patterns of sentiment scores are different between two platforms. For BlueSky, the sentiment scores are more concentrated around zero, which indicate a more neutral attitude in the posts. For Mastodon, the sentiment score distribution is broader, with some posts receiving high positive sentiment scores. This suggests that discussion on Mastodon contain more emotional expression, and posts contain more positive language. Furthermore, there are a small number of posts on both platforms in the range of -0.2 and -0.4, indicate that it does not present a strongly negative attitude. Thus, Australians exhibit slightly different attitudes towards conflict on BlueSky and Mastodon platforms, but overall, they remain close to neutral.

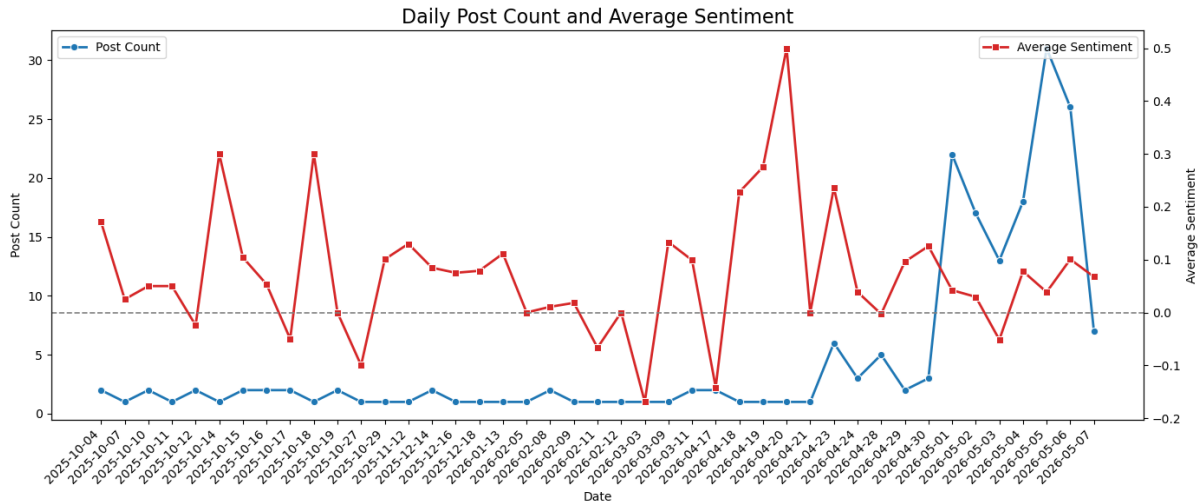


Figure 6: Daily post count and average sentiment

Figure 6 shows the changes in the number of posts and average sentiment score over time. The blue line represents the number of posts crawling every day. The red line represents the average sentiment score every day. This plot shows in early stages, only few posts publish on most days. In this stage, the average sentiment score fluctuated significantly from -0.2 to 0.5. This may because the small sample size, the few posts with strong sentiment will affect the average sentiment score. Start around May 2026, the number of posts increase significantly, this makes the average sentiment score fluctuate between -0.05 and 0.1, the overall sentiment became more stable, indicate that most of the added posts are closer to neutral in their attitude.

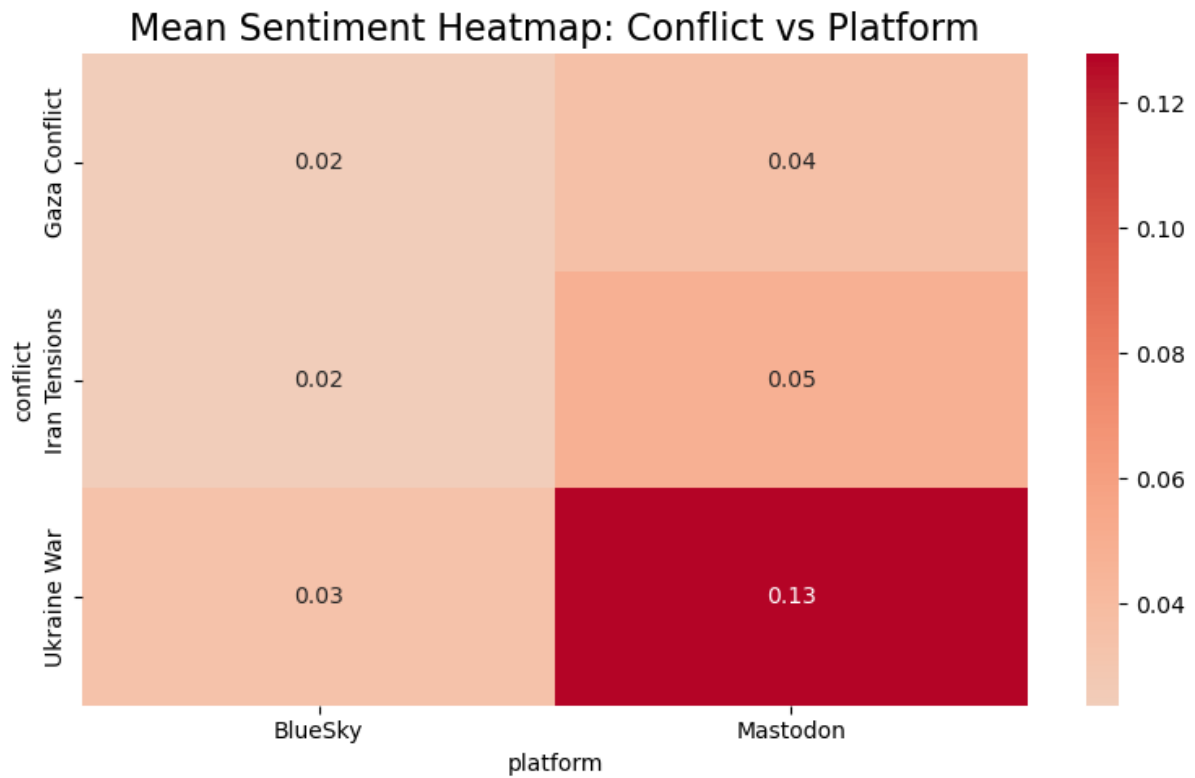


Figure 7: Mean sentiment heatmap by conflict and platform

Figure 7 is a heatmap comparing the average sentiment scores for the three conflicts on BlueSky and Mastodon, showing that all values are slightly above 0, indicating a slightly positive overall sentiment in the collected samples. Mastodon's average sentiment score is 0.13 and for BlueSky's was 0.03. Therefore, Mastodon average sentiment score was higher than BlueSky. In both Gaza Conflict and Iran Tensions, Australians' average sentiment was closer to neutral on both platforms. Thus, the sentiment differences between the two platforms were mainly focused on the Ukraine War.

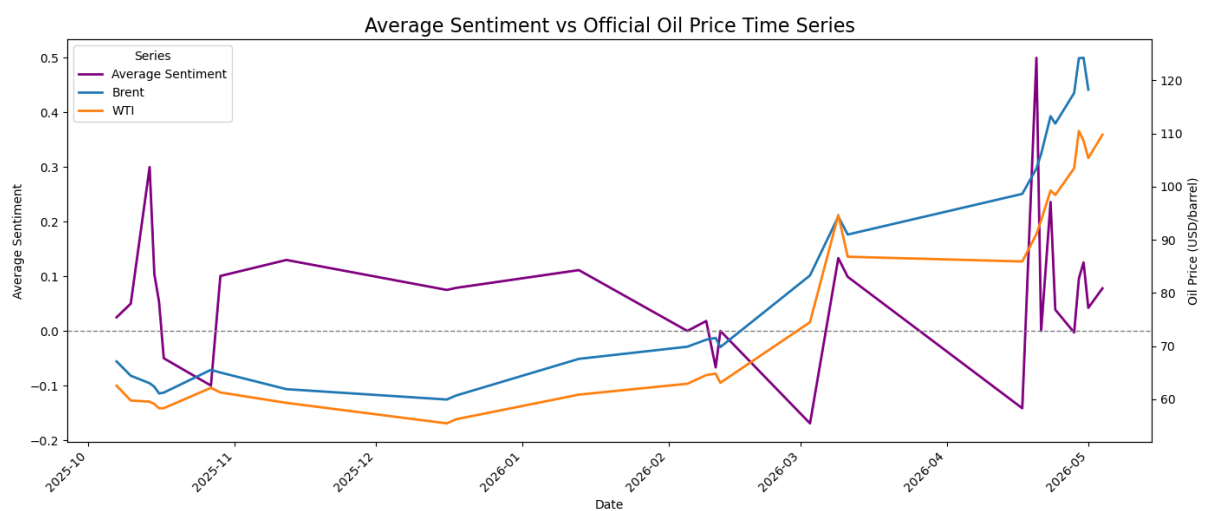


Figure 8: Average sentiment and official oil price time series

Figure 8 compares Brent and WTI oil prices and average sentiment scores over time. Brent and WTI show an

increased trend. From Oct 2025 to Feb 2026, oil prices between \$60 and \$75. In May 2026, reaching \$120 for Brent and \$110 for WTI. For the sentiment fluctuated around a neutral level. Starting in March 2026, sentiment score will first decreased, then rise to its most positive, it is close to 0.5, finally trending towards neutral. Therefore, as oil prices increased, the sentiment score did not show a increased trend. So, oil prices changes not influence changes in sentiment.

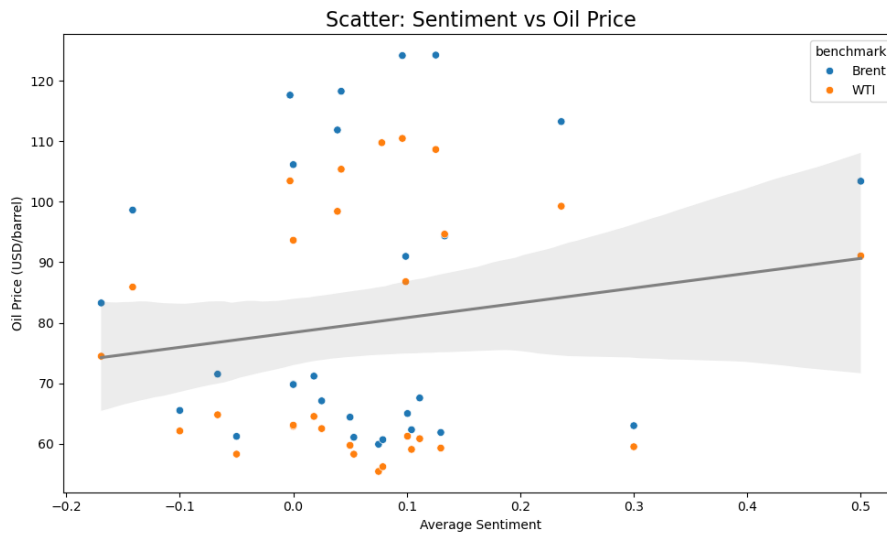


Figure 9: Scatter plot of sentiment and oil price

Figure 9 is a scatter plot of sentiment and oil prices. As oil prices generally trend upward, the average sentiment among Australians is higher. The scatter plot shows that Brent is higher than WTI, the most of the blue points are above the orange points, they do not have clear linear structure. Therefore, the correlation between Brent, WTI and sentiment is not strong.

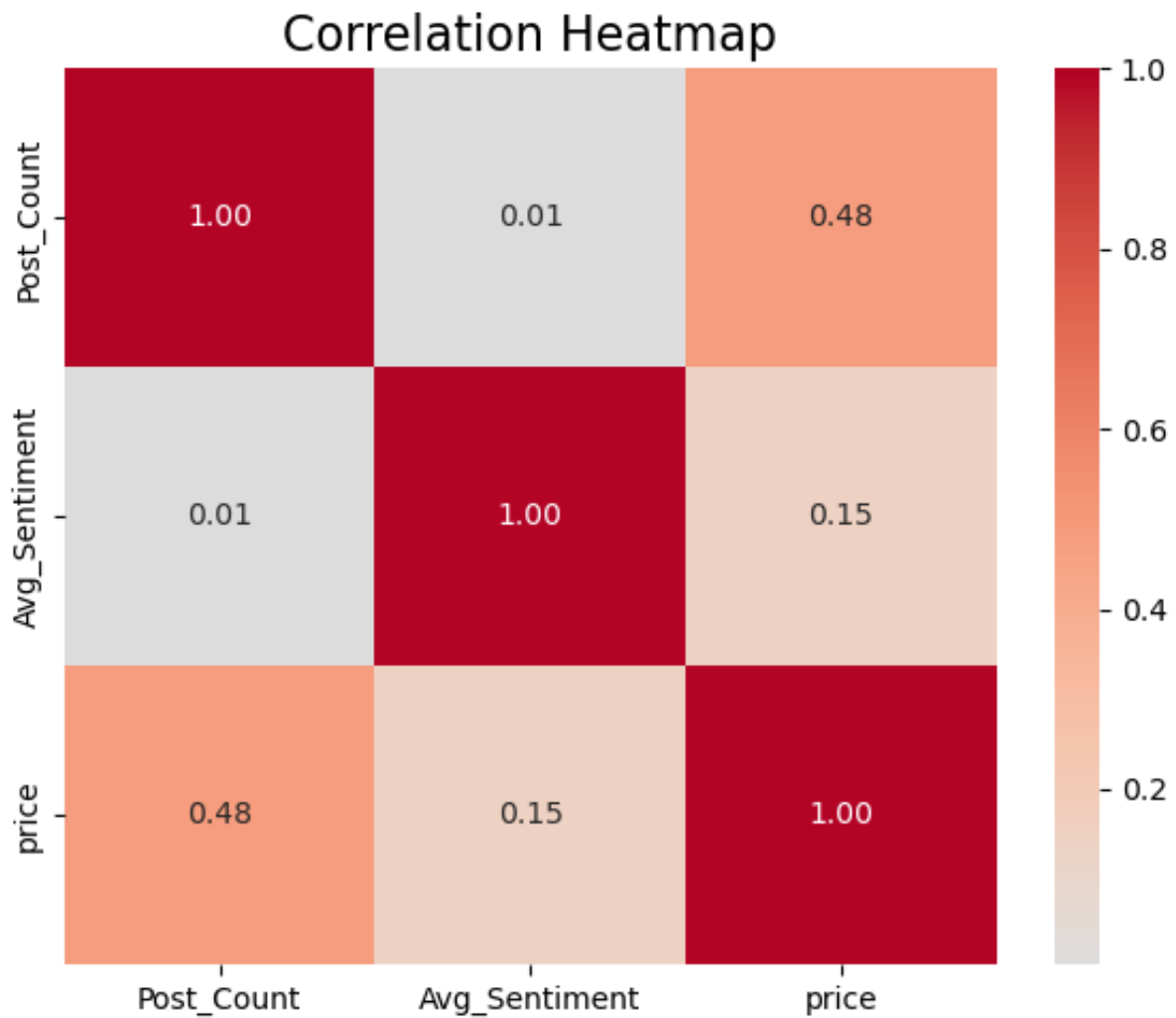


Figure 10: Correlation heatmap for post count, sentiment and oil price

Figure 10 is a heatmap for the correlation between the number of posts, average sentiment and oil prices. Between the number of posts and oil prices is a moderate positive correlation of 0.48, it show that periods of higher oil prices in this sample were often accompanied by more discussion among Australians. The correlation coefficient between average sentiment and oil prices is only 0.15, it is a weak positive correlation. So it indicate a weak correlation between sentiment changes and oil price trends. The correlation coefficient between the number of posts and average sentiment is 0.01, it is no correlation, this suggests that increased discussion does not lead to a significant shift in people's attitudes.

5 Platform Deployment

5.1 Docker and Local Development

To set up the development environment, we used Docker Compose to run Elasticsearch 8.12.2 as a single-node service on port 9200. To ensure the controllability of the prototype environment, we disabled security features. The backend container is based on Python 3.11-slim, and its dependencies are managed via a requirements.txt file. The backend code was copied to the /app/backend directory and run on port 8000 using uvicorn backend.backendmain:app. This approach ensures that features such as data collection, Elasticsearch storage, API endpoints, and notebook visualizations are validated before migrating to the cloud.

The same backend image was also used in the Kubernetes deployment. The Kubernetes API specification uses the GitLab Container Registry image: registry.gitlab.unimelb.edu.au:5005/chenxuanjin/comp90024_team_8/yaojinw-main-patch-25838:latest'. Both the deployment of the API and scheduled tasks use the same image. Therefore, these two components share the same Python dependencies and backend source files. This helps to minimize the environmental differences between local testing, API deployment, and scheduled tasks.

5.2 Kubernetes and NeCTAR/MRC Cloud

This Kubernetes deployment was implemented within the "comp90024" namespace of the NeCTAR/MRC cloud cluster. Elasticsearch accesses internally through the ClusterIP service on port 9200, while the FastAPI backend connects to it via 'ES_URL=http://elasticsearch:9200'. To facilitate future maintenance, the Elasticsearch and FastAPI backends are deployed as separate services using repository images. Elasticsearch is mainly responsible for data storage and search, FastAPI acts as the API access layer. The cloud testing process was carried out on Kubernetes. After the data was extracted to the cloud Elasticsearch service, tested multiple ports. The screenshot below shows the test results.

```
PS C:\Users\wjj56\Desktop\COMP90024ASSI2\melbourne-university-project> openstack coe cluster list
```

uuid	name	keypair	node_count	master_count	status	health_status
86f9b243-60fb-46d5-a4b8-991319149511	comp90024	comp90024-key	1	1	CREATE_COMPLETE	HEALTHY

Figure 11: MRC cluster status: CREATE_COMPLETE and HEALTHY

```
PS C:\Users\wjj56\Desktop\COMP90024ASSI2\kubeconfig> kubectl get nodes
```

NAME	STATUS	ROLES	AGE	VERSION
comp90024-4o5axyzfuawc-control-plane-xfkpl	Ready	control-plane	2d23h	v1.35.3
comp90024-4o5axyzfuawc-default-worker-wxczm-h5gfl	Ready	<none>	2d23h	v1.35.3

Figure 12: Kubernetes node readiness

```
PS C:\Users\wjj56\Desktop\COMP90024ASSI2\melbourne-university-project> kubectl get pods -n comp90024
```

NAME	READY	STATUS	RESTARTS	AGE
comp90024-api-656c54fd56-w7npl	1/1	Running	0	4m33s
elasticsearch-564ff6d77d-7gzz4	1/1	Running	0	13m

Figure 13: Kubernetes pod status in the comp90024 namespace

health	status	index	uuid	pri	rep	docs.count	docs.deleted	store.size	pri.store.size	dataset.size
yellow	open	market_prices	SZJ-yLnBR0S9j10s1fz15Q	1	1	2175	0	118.9kb	118.9kb	118.9kb
yellow	open	social_posts	Ix4YQV8UIxe5WWG1IKgHjQ	1	1	195	0	934.3kb	934.3kb	934.3kb

Figure 14: Elasticsearch index counts for social_posts and market_prices

```
{
  "message": "COMP90024 Social Media Analytics API is running",
  "elasticsearch_url": "http://elasticsearch:9200",
  "elasticsearch_connected": true,
  "social_index": "social_posts",
  "market_index": "market_prices",
  "social_post_count": 195,
  "market_price_count": 2175,
  "columns": [
    "platform",
    "date",
    "text",
    "author",
    "upvotes",
    "url",
    "sentiment_score",
    "conflict",
    "clean_text"
  ]
}
```

Figure 15: FastAPI root response with Elasticsearch connection and record counts

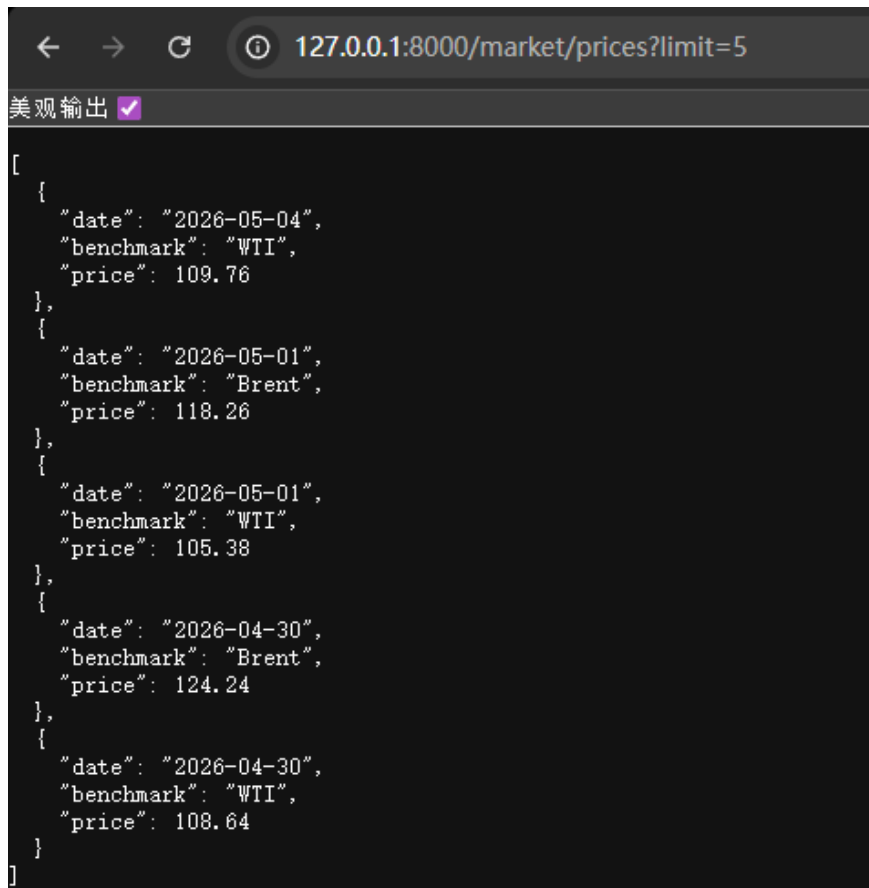
COMP90024 Social Media Analytics API 0.1.0 OAS 3.1

/openapi.json

default

- GET / Root
- GET /health Health Check
- GET /posts Get Posts
- GET /posts/search Search Posts
- GET /stats/platform Platform Stats
- GET /stats/conflict Conflict Stats
- GET /stats/sentiment Sentiment Stats
- GET /stats/platform-sentiment Platform Sentiment Stats
- GET /stats/timeline Timeline Stats
- GET /stats/keywords Keyword Stats
- GET /stats/summary Summary Stats
- GET /market/prices Get Market Prices
- GET /market/latest Latest Market Price

Figure 16: Swagger documentation for REST API endpoints



```
[
  {
    "date": "2026-05-04",
    "benchmark": "WTI",
    "price": 109.76
  },
  {
    "date": "2026-05-01",
    "benchmark": "Brent",
    "price": 118.26
  },
  {
    "date": "2026-05-01",
    "benchmark": "WTI",
    "price": 105.38
  },
  {
    "date": "2026-04-30",
    "benchmark": "Brent",
    "price": 124.24
  },
  {
    "date": "2026-04-30",
    "benchmark": "WTI",
    "price": 108.64
  }
]
```

Figure 17: Sample official oil price records returned by /market/prices



```
[
  {
    "platform": "Mastodon",
    "post_count": 98
  },
  {
    "platform": "BlueSky",
    "post_count": 97
  }
]
```

Figure 18: Platform count endpoint returned by /stats/platform

5.3 Fission and Scheduled Harvesting

Our system mainly uses Fission for scheduled data ingestion, with Kubernetes' CronJob serving as a backup. The platform components of Fission are located in the fission namespace. For functions, environments and

timers are placed in the default namespace. The function code is in `fission/trigger_ingestion.py`, and its name is `trigger-ingestion`.

This function does not run the entire ingestion process by itself. Instead, it sends a POST request to the FastAPI service within the cluster. Once the API receives the request, it will call the functions `ingest_social_posts()` and `ingest_market_prices()`, and then write the updated data to Elasticsearch.

The task that is triggered at regular intervals is called "ingestion-daily", and it uses Fission's timer. Additionally, the `k8s/ingest-cronjob.yaml` file in the repository defines a native Kubernetes backup configuration. In case there is a problem with Fission that requires debugging or restoration, we can directly run the same set of ingestion scripts from the backend Docker image. This way, we can achieve the effect of saving resources normally and quickly recovering in case of problems.

It should be noted that currently, it is the Fission scheduled task that is actually running. The CronJob merely exists as an optional backup configuration file and is not a second running timer.

The following screenshots are the test results.

```
PS C:\Users\wjj56\Desktop\COMP90024ASSI2\melbourne-university-project> kubectl get pods -n fission
NAME                                READY   STATUS    RESTARTS   AGE
buildermgr-76d4f6fdbb-tdk9v         1/1     Running   0           134m
executor-74fcc6c7d4-xljbh           1/1     Running   0           134m
kubewatcher-f8fb6cb69-phm5b         1/1     Running   0           134m
mqtrigger-keda-5fbbdd98b9-qrdq6     1/1     Running   0           134m
router-5b8d98d477-45vqh             1/1     Running   0           134m
storagesvc-5c9575cc69-nbg96         1/1     Running   0           134m
timer-dc94f9bbc-mvgr7              1/1     Running   0           134m
webhook-84cf8c4488-kg9ws            1/1     Running   0           134m
PS C:\Users\wjj56\Desktop\COMP90024ASSI2\melbourne-university-project> |
```

Figure 19: Fission namespace pods running

```
PS C:\Users\wjj56\Desktop\COMP90024ASSI2\melbourne-university-project> fission check
fission-services
-----
✓ executor is running fine
✓ router is running fine
✓ storagesvc is running fine
✓ webhook is running fine

fission-version
-----
✓ fission is up-to-date
PS C:\Users\wjj56\Desktop\COMP90024ASSI2\melbourne-university-project> |
```

Figure 20: Fission health check

```
PS C:\Users\wjj56\Desktop\COMP90024ASSI2\melbourne-university-project> fission function list --namespace default
NAME      ENV  EXECUTORTYPE  MINSIZE  MAXSCALE  MINCPU  MAXCPU  MINMEMORY  MAXMEMORY  SECRETS  CONFIGMAPS  NAMESPACE
trigger-ingestion python poolmgr 0 0 0 0 0 0 0 0 default
PS C:\Users\wjj56\Desktop\COMP90024ASSI2\melbourne-university-project> |
```

Figure 21: Fission function list showing trigger-ingestion

```
PS C:\Users\wjj56\Desktop\COMP90024ASSI2\melbourne-university-project> fission timer list --namespace default
NAME          CRON          FUNCTION_NAME  METHOD  SUBPATH
ingestion-daily 0 0 0 * * * trigger-ingestion POST    /
PS C:\Users\wjj56\Desktop\COMP90024ASSI2\melbourne-university-project> |
```

Figure 22: Fission timer list showing ingestion-daily

```
PS C:\Users\wjj56\Desktop\COMP90024ASSI2\melbourne-university-project> fission function test --name trigger-ingestion --namespace default
{"status": "completed", "message": "Ingestion completed successfully", "elasticsearch_url": "http://elasticsearch:9200", "social_index": "social_posts", "market_index": "market_prices"}
PS C:\Users\wjj56\Desktop\COMP90024ASSI2\melbourne-university-project> |
```

Figure 23: Manual Fission function test returning completed ingestion

6 Technology Selection and Evaluation

The choice of Python was made because the team is familiar with it, and it has mature libraries to support API access, data cleaning, sentiment analysis, Elasticsearch integration, and Notebook visualization. Pandas offers table processing, BeautifulSoup extracts text from HTML content, TextBlob provides lightweight sentiment scores, FastAPI offers a REST backend, and Jupyter Notebook provides an interactive analysis front-end.

This project uses Elasticsearch because our data is not simply a simple fixed table. The social media records contain various fields, such as the post text, date, platform, author, URL, emotion score, conflict tags and the cleaned text. The oil price data is also a time-series data that changes over time, including two types of prices: Brent and WTI. Therefore, it is a natural design to store the processed social media data in the "social_posts" index and the oil price data in the "market_prices" index.

Compared to simply saving the results as CSV files, Elasticsearch provides a more practical data storage layer for the project. After data collection and cleaning are completed, the records can be written to Elasticsearch, which can then be further utilized by the backend. FastAPI will read the indexed data from Elasticsearch and then provide it to Jupyter Notebook through REST endpoints. In this way, the Notebook no longer needs to directly rely on local CSV files, nor does it need to repeat the entire data collection process. The entire system will also be more apparent, as data collection, data storage, API access, and visualization are respectively handled by different parts.

Kubernetes offers a real cloud deployment model and also has the CronJob option for scheduled tasks. At the same time, namespaces can isolate different projects to avoid interference among them; the service name serves as a unified address (such as elasticsearch:9200), making it convenient to switch between development and testing environments; the YAML file, combined with CI/CD, enables reproducible deployment, eliminating the need for manual operations each time by SSHing in.

However, there may also be issues such as YAML errors, authentication problems with the image repository, kubeconfig configuration problems, service exposure problems, insufficient cluster quotas, and version incompatibilities. But this trade-off is worthwhile for the cloud-based analysis system.

Adding an additional layer of serverless to Fission on Kubernetes offers several benefits. The main advantage is that the periodic ingestion becomes a triggered function rather than a long-running process. This results in a modular call point for the pipeline, allowing the ingestion trigger to be tested manually separately. Moreover, adding new collectors or query functions can be expanded in the same way in the future.

7 Discussion

7.1 Suitability of the Cloud-native Architecture

This architecture divides the functions of data collection, preprocessing, storage, API, and presentation into different levels. This approach prevents the Notebook from undertaking too many responsibilities. When adding new fields or new data sources, only the corresponding layer needs to be modified, without having to make comprehensive changes.

In terms of team collaboration, once the plan and API protocol are agreed upon, many subsequent parts can be carried out simultaneously. This significantly improves the efficiency of collaboration among the teams.

However, its drawback is that the integration cost is relatively high. In the subsequent integration process, there will be more areas that need to be checked, and all these parts need to be consistent with the actual operating system.

7.2 Kubernetes and MRC Trade-offs

The greatest advantage of using Kubernetes on MRC lies in its ease of operation. Different projects can be managed separately. This also makes it easy for us to distinguish between development, testing, and production environments.

However, the cost is that this will result in more complex configuration files. Issues such as image repository authentication, kubeconfig, OpenStack tools, insufficient node resources, version mismatch, and incorrect Elasticsearch configuration all need to be addressed. We will discuss these issues in detail in the subsequent challenge section.

7.3 Fission, CronJob and Scheduled Ingestion

Fission serves as the primary timer, while we also retain CronJob as a backup solution. The advantage of this design is that by calling the /ingest/run endpoint of FastAPI from Fission. Compared to using only CronJob, this approach is more modular and aligns better with the event-driven design philosophy.

In our design, the Fission function is only responsible for sending one HTTP request and does not directly run the complete data ingestion logic. Therefore, it has fewer dependencies and starts up faster.

The timing trigger is handled by the cluster controller, which is more stable than relying on the Notebook process. At the same time, the function and timer can be updated independently without need to restart the backend.

The drawback of this design is that the operation and maintenance become more complex. Using Fission need additional maintenance. During the debugging process, more components are involved. So, we have retained CronJob as a backup solution. In practical application, fission is only used as a trigger. The actual data collection and indexing logic still remain in the FastAPI backend. This approach makes the function more lightweight.

7.4 Elasticsearch and API Design Trade-offs

Compare with CSV files, Elasticsearch offers more powerful indexing, searching and aggregation ability, it also requires more granular management of memory usage, field mapping, service health and data persistence.

In the current system, some FastAPI endpoints first retrieve documents from Elasticsearch. Then use pandas to group the data and calculate basic statistics. This approach is unsuitable as the dataset grows further. Future

development should utilize Elasticsearch's aggregation capabilities more to reduce data transfer volume and improve query performance.

If the system continues to expand, more computational tasks should be offloaded to Elasticsearch queries. For example, the system can use methods such as terms aggregation, date histograms, and paginated queries. These methods can reduce the amount of data transferred to the FastAPI while improving query efficiency. Therefore, Elasticsearch offers more powerful data backend capabilities compared to static CSV files. FastAPI hides the complexity of Elasticsearch and provides a stable REST interface for Jupyter Notebook.

7.5 Data Quality, Australia Relevance and NLP Limitations

Our system focuses solely on content from Australian social media platforms. Data is filtered by retrieving relevant keywords, account information, context, and mentions of Australia in the text. This limits the scope of the results.

The collected data may contain diverse content types. For example, some posts may be related to news events, while others may simply mention Australia. Therefore, the findings of this study should not be considered representative of the views of all Australians, but rather reflect the Australia-related data collected by our system.

In sentiment analysis, the use of TextBlob and rule-based methods also limits the accuracy of the analysis. TextBlob may be biased when handling sarcasm, irony, quotes, political context, multilingual posts, negative expressions, and conflict-related vocabulary.

7.6 Scalability, Fault Tolerance and Future Improvements

The system can be scaled up in multiple ways. New data collectors can be added, and more official datasets can be integrated. Moreover, the number of API replicas in Kubernetes can be scaled up, and health probes can be integrated to enhance monitoring. Future development will also enable even more extensive use of Elasticsearch's aggregation features. Fission also paves the way for future expansion. New fission functions can be created for different collectors, processors, or query tasks. This makes it possible to deploy and test each component separately, which simplifies system maintenance. We can also consider a more event-driven design method for the future. For instance, Redis or other message brokers could be placed between data collection and indexing, which would separate the collection process from the storage one.

Our project provides some fault tolerance. Kubernetes can recover from failed APIs or Elasticsearch Pods. Fission triggers can run independently, and when Elasticsearch fails, the backend returns an empty data structure instead of crashing. Furthermore, If the backup path is enabled, the optional Kubernetes CronJob provides basic retry support. It uses restartPolicy: OnFailure. It also uses backoffLimit: 2. These settings allow the job to restart or retry when ingestion fails. Our main limitation is Elasticsearch. The current system uses a single-node Elasticsearch instance and does not use persistent storage. If the Elasticsearch Pod is rescheduled, data may be lost. The system also has no replication. Therefore, it cannot provide high availability.

Another step is to deduplicate the planned data ingestion runs. Although duplicates are removed from each collected dataset during the preprocessing phase, Elasticsearch's write path is not always used to compare new records with documents stored in previous runs. Therefore, social media posts or market records may be duplicated, potentially skewing subsequent statistics. The design should use deterministic document IDs based on URL, platform, and date, or implement logic to update or replace documents before writing them to Elasticsearch.

8 Error Handling, Security and Reliability

This implementation incorporates various application-level error handling mechanisms. Although these protections are mainly located at the API request and data conversion levels, rather than being part of a comprehensive fault-tolerance design. The Mastodon and Bluesky collection loops will capture the request errors for each query and continue to run without terminating the entire collection process. If the login of Bluesky returns an unsuccessful status, an empty DataFrame will be returned; for the loading of EIA oil prices, multiple possible header rows will be attempted. If the available table cannot be parsed, an empty DataFrame with the expected schema will be returned. The date and numeric fields will be converted using coercion to prevent crashes caused by format errors in the records.

The backend also includes defensive handling for null data or unreachable Elasticsearch states. The social posts and market price loaders will catch failures in Elasticsearch reads and return empty DataFrames with the expected columns. The /health endpoint reports whether Elasticsearch is accessible, while the root endpoint provides information such as the Elasticsearch URL, connection status, index names, and the number of records. The aggregation endpoint checks whether the dataset is empty before grouping, and the API data returned to the Notebook will also be converted into JSON-safe records. The API-based Notebook/test workflow includes health-check cells, and the main visualization Notebook also uses API requests and status checks when loading posts and market prices.

ubernetes offers basic restart functionality for both the deployed API and Elasticsearch pods. Fission offers a readily deployable and independently testable scheduled invocation path. This is done by calling the FastAPI endpoint /ingest/run, which reuses the same ingestion logic as the direct ingestion script. The optional Kubernetes CronJob manifest offers a backup scheduled ingestion path, with built-in retry and overlap policies using , restartPolicy: OnFailure, , backoffLimit: 2, and , concurrencyPolicy: Forbid. The following reliability enhancements were implemented, including readiness and liveness probes for the API, single node Elasticsearch configuration, and manual endpoint checks for notebook-based API tests are provided.

The credential handling is kept basic for this academic prototype, and is not production-grade. Credentials are managed through local credential files during development and via Kubernetes Secret mounts in the cloud. The API and ingestion pods read the paths of Mastodon credential files from environment variables and mount the mastodon-credentials Secret at runtime, which keeps the container image independent of local machine paths. However, Bluesky credentials should also be moved completely into environment variables or Kubernetes Secrets, rather than relying on default values coded within the application. In short, the prototype has only basic security measures: Elasticsearch security is turned off, the API does not have any frontend authentication, and CORS is open for testing on notebooks. These choices are acceptable for a controlled coursework prototype but should be strengthened for any production deployment.

9 Challenges and Solutions

One of the early challenges was to ensure consistency among the local development environment, the backend, the data import script, and Elasticsearch. The import script could only be effectively tested after Docker Compose successfully started Elasticsearch and the two indexes (social posts and market prices) were successfully created. During the testing process, the team also encountered a problem that the Elasticsearch client and the server were incompatible: the local Elasticsearch server version was 8.12.2, but the installed Python Elasticsearch client would generate request headers for a newer version, causing the import script to fail even when the Elasticsearch service itself was running normally when writing data to Elasticsearch. The team solved this problem by keeping the version of the Python Elasticsearch client consistent with the server version and fixing the local startup process: first start Docker and Elasticsearch, then create or check the indexes, then run the import script, and finally test the FastAPI endpoint. This way, the local process became more repeatable and stable before being migrated to the cloud.

In terms of Kubernetes deployment, the backend image needs to be tagged using the image repository address of the project's GitLab first, and then pushed to the repository. Only then can K8s pull and run the image from the cloud. Otherwise, the Pod will not be able to obtain the image. The ES_URL is incorrect. Even if the Pod starts successfully, it will not be able to connect to Elasticsearch.

We have also encountered situations where the service was operating normally but there was no data in the cloud. The Pod status of both API and Elasticsearch shows "Running", but this does not mean that there are data in Elasticsearch. The actual approach is as follows: Run a data ingestion process for the Elasticsearch on the cloud to confirm that the two indices, social_posts and market_prices, have been created and data has been written into them. Then test several interfaces of the API (root path, /posts, /market/prices, /stats/platform) and the Swagger documentation. After rechecking the local files, the connection to Elasticsearch is confirmed.

The stability of data under scheduled intake and repeated operation also needs to be paid attention to. This Fission link requires multiple layers of verification: check the status of the platform Pods in the Fission namespace, run the Fission check to confirm that the executor, router, storagesvc, and webhook are all functioning properly, confirm the existence of the trigger-ingestion function, confirm the existence of the ingestion-daily timer, and then manually execute the function test once. Only after all these steps are completed successfully can it be determined that the scheduled function can properly call the /ingest/run interface on the backend.

Currently, there is still a stability issue: The current ingestion process does not perform idempotently when writing to Elasticsearch. That is to say, if the timer is repeatedly executed without using a determined document ID, upsert logic, index refresh, or cross-run deduplication, the already collected post and price records may be repeatedly appended. Therefore, in this submitted analysis, the Notebook uses the dataset after a single run as a stable snapshot. In a production environment, a deduplication or upsert mechanism must be added; otherwise, repeated runs will cause the statistical results to deviate from the real data.

10 Project Summary

This project details a cloud-based workflow for exploring the correlation between social media activity in Australia, public sentiment related to major geopolitical conflicts, and global oil price fluctuations. The system collects posts from Mastodon and Bluesky, extracts EIA oil price data, cleans and enhances the records, stores them in Elasticsearch, and makes them available to a Notebook frontend through FastAPI. The core components are deployed on Kubernetes. The system also incorporates a Fission trigger function, which utilizes the `/ingest/run` endpoint to take advantage of existing data ingestion logic and keeps a planned ingestion path as a backup.

Based on the figures, the crawled dataset shows that the discussion for three conflict themes are uneven in the Australian social media. The Ukraine War receives the highest discussion volume, while Iran Tensions receives the lowest. Then the sentiment is generally close to neutral on both platforms, although Mastodon shows slightly more positive and more varied sentiment than BlueSky. Finally, the oil prices have a stronger relationship with post count than with average sentiment. Therefore, the main finding is that conflict-related discussion activity varies clearly by topic and is moderately associated with oil prices, while emotional sentiment remains largely neutral and only weakly related to oil price movements.

Recent analysis reveals that the overall activity across the platform is even, but the focus on the Ukraine war is more pronounced than its relationship with the average public sentiment. The main limitations are that the sample is keyword-distributed and repeated ingestion from Elasticsearch is not yet fully idempotent. These limitations do not reduced the project's value. They set the scope of the current findings and point out areas for future work with larger collections, cross-run deduplication and more complex time series analysis.

References

- Australian Competition and Consumer Commission. (n.d.). Inquiries and consultations [Retrieved May 17, 2026]. <https://www.accc.gov.au/inquiries-and-consultations>
- Bluesky. (n.d.). Bluesky documentation [Retrieved May 17, 2026]. <https://docs.bsky.app/>
- Elastic. (n.d.). Elasticsearch documentation [Retrieved May 17, 2026]. <https://www.elastic.co/guide/>
- FastAPI. (n.d.). Fastapi documentation [Retrieved May 17, 2026]. <https://fastapi.tiangolo.com/>
- Fission. (n.d.). Fission documentation [Retrieved May 17, 2026]. <https://fission.io/docs/>
- Kilian, L., Plante, M. D., Richter, A. W., & Zhou, X. (2026). *The impact of the 2026 iran war on u.s. inflation: A scenario analysis* (tech. rep. No. Working Paper No. 2609). Federal Reserve Bank of Dallas. <https://doi.org/10.24149/wp2609>
- Kubernetes. (n.d.). Kubernetes documentation [Retrieved May 17, 2026]. <https://kubernetes.io/docs/>
- Mastodon. (2025, June). Getting started with the api. <https://docs.joinmastodon.org/client/intro/>
- TextBlob. (n.d.). Textblob: Simplified text processing [Retrieved May 17, 2026]. <https://textblob.readthedocs.io/>
- U.S. Energy Information Administration. (n.d.). Spot prices for crude oil and petroleum products [Data set. Retrieved May 17, 2026]. https://www.eia.gov/dnav/pet/xls/PET_PRI_SPT_S1_D.xls